

Data types in mysql

- Numeric
- Date and Time
- String Types.

Let us now discuss them in detail.

Numeric Data Types

MySQL uses all the standard ANSI SQL numeric data types, so if you're coming to MySQL from a different database system, these definitions will look familiar to you. The following list shows the common numeric data types and their descriptions –

- **INT** – A normal-sized integer that can be signed or unsigned. If signed, the allowable range is from -2147483648 to 2147483647. If unsigned, the allowable range is from 0 to 4294967295. You can specify a width of up to 11 digits.
- **TINYINT** – A very small integer that can be signed or unsigned. If signed, the allowable range is from -128 to 127. If unsigned, the allowable range is from 0 to 255. You can specify a width of up to 4 digits.
- **SMALLINT** – A small integer that can be signed or unsigned. If signed, the allowable range is from -32768 to 32767. If unsigned, the allowable range is from 0 to 65535. You can specify a width of up to 5 digits.
- **MEDIUMINT** – A medium-sized integer that can be signed or unsigned. If signed, the allowable range is from -8388608 to 8388607. If unsigned, the allowable range is from 0 to 16777215. You can specify a width of up to 9 digits.
- **BIGINT** – A large integer that can be signed or unsigned. If signed, the allowable range is from -9223372036854775808 to 9223372036854775807. If unsigned, the allowable range is from 0 to 18446744073709551615. You can specify a width of up to 20 digits.
- **FLOAT(M,D)** – A floating-point number that cannot be unsigned. You can define the display length (M) and the number of decimals (D). This is not required and will default to 10,2, where 2 is the number of decimals and 10 is the total number of digits (including decimals). Decimal precision can go to 24 places for a FLOAT.
- **DOUBLE(M,D)** – A double precision floating-point number that cannot be unsigned. You can define the display length (M) and the number of decimals (D). This is not required and will default to 16,4, where 4 is the number of decimals. Decimal precision can go to 53 places for a DOUBLE. REAL is a synonym for DOUBLE.
- **DECIMAL(M,D)** – An unpacked floating-point number that cannot be unsigned. In the unpacked decimals, each decimal corresponds to one byte. Defining the display length (M)

Date and Time Types

The MySQL date and time datatypes are as follows –

- **DATE** – A date in YYYY-MM-DD format, between 1000-01-01 and 9999-12-31. For example, December 30th, 1973 would be stored as 1973-12-30.
- **DATETIME** – A date and time combination in YYYY-MM-DD HH:MM:SS format, between 1000-01-01 00:00:00 and 9999-12-31 23:59:59. For example, 3:30 in the afternoon on December 30th, 1973 would be stored as 1973-12-30 15:30:00.
- **TIMESTAMP** – A timestamp between midnight, January 1st, 1970 and sometime in 2037. This looks like the previous DATETIME format, only without the hyphens between numbers; 3:30 in the afternoon on December 30th, 1973 would be stored as 19731230153000 (YYYYMMDDHHMMSS).
- **TIME** – Stores the time in a HH:MM:SS format.
- **YEAR(M)** – Stores a year in a 2-digit or a 4-digit format. If the length is specified as 2 (for example YEAR(2)), YEAR can be between 1970 to 2069 (70 to 69). If the length is specified as 4, then YEAR can be 1901 to 2155. The default length is 4.

String Types

Although the numeric and date types are fun, most data you'll store will be in a string format. This list describes the common string datatypes in MySQL.

- **CHAR(M)** – A fixed-length string between 1 and 255 characters in length (for example CHAR(5)), right-padded with spaces to the specified length when stored. Defining a length is not required, but the default is 1.
- **VARCHAR(M)** – A variable-length string between 1 and 255 characters in length. For example, VARCHAR(25). You must define a length when creating a VARCHAR field.
- **Nvarchar** stores Unicode or Non-English character data types, and it can contain a maximum of 4000 characters. It supports ASCII values as well as special characters. To support multiple languages, nvarchar is a must.
- **BLOB or TEXT** – A field with a maximum length of 65535 characters. BLOBs are "Binary Large Objects" and are used to store large amounts of binary data, such as images or other types of files. Fields defined as TEXT also hold large amounts of data. The difference between the two is that the sorts and comparisons on the stored data are **case sensitive** on BLOBs and are **not case sensitive** in TEXT fields. You do not specify a length with BLOB or TEXT.

- **TINYBLOB or TINYTEXT** – A BLOB or TEXT column with a maximum length of 255 characters. You do not specify a length with TINYBLOB or TINYTEXT.
- **MEDIUMBLOB or MEDIUMTEXT** – A BLOB or TEXT column with a maximum length of 16777215 characters. You do not specify a length with MEDIUMBLOB or MEDIUMTEXT.
- **LOB or LONGTEXT** – A BLOB or TEXT column with a maximum length of 4294967295 characters. You do not specify a length with LOB or LONGTEXT.
- **ENUM** – An enumeration, which is a fancy term for list. When defining an ENUM, you are creating a list of items from which the value must be selected (or it can be NULL). For example, if you wanted your field to contain "A" or "B" or "C", you would define your ENUM as ENUM ('A', 'B', 'C') and only those values (or NULL) could ever populate that field.

Accid	Custid	Type	balance	Rel.mgr	empno
1234	1000	Saving	234567	Kavita	10
1235	1000	current	254567	Kavita	10
1236	1000	demat	274567	Kavita	10
1237	1001	saving	333333	Rohit	20

Custid	Cname	addr
1000	Kishori	Aundh
1001	Rajan	Aundh
1002	Ashish	Baner

Emp(empid,ename,adharcadno,email, mobile,pan no,passportno,address)

982342341234

982342341234

Types of keys

primary key	minimal set of columns which identifies the row uniquely is called as primary key There will be only one primary key if primary key contains single column then it is called as simple primary key if primary contains more than one column, then it is called as composite primary key
candidate key	all possible minimal set of columns which identifies the row uniquely is called as candidate key
surrogate key	if primary key has multiple columns and if managing the data is difficult, then we add one extra column to make the row unique, is called as surrogate key
alternate key	all candidate keys which are not chosen as primary key are called as alternate key

super key	any combination of columns which identifies the row uniquely is called as super key
foreign key	it is a column which refers column of primary key or unique column of other table or of same table, then it is called as foreign key In a table there can be more than one foreign key
Unique key	If you need that data should be unique in the column, then it is called as unique key. It can contain many null values

examples

1. in room booking table

room(rno,rname,rloc,rdesc)

booking(roomno,rname,date of booking, custid, price)

primary key---roomno+date of booking. Or may add bookingid as sarogate key

candidate key--- roomno+date of booking

alternate key---no alternate key

super key ---→ roomno+date of booking , roomno+date of booking+rname,
roomno+date of booking+price, roomno+date of booking+custid,+ roomno+date of
booking+custid+price

foreign key-----**roomno** of booking table is a foreign key, it should refer roomno in room
table

custid in booking table is a foreign key, it should refer customer table's customerno

2. doctor patient appointment system

doctor(drid,dname,specialization)

appointment(drid, pid, date,time,amt,prescription)

patient(ptid,pname,address.mobile)

primary key-→pid+date+time

sarogate key---→ you may add extra column bookingid, which may become surrogate
key and can be used as primary key

candidate keys-→ pid+date+time, drid+date+time

alternate key-→ drid+date+time

foreign key --→ drid is foreign key, refers to doctor table drid

pid is foreign key, refers to patient table ptid

constraints on the table

primary key	it does not allow duplicate values and null values It is table level constraints, there will be only one primary key in the table
unique key	it allows only unique values if it is not null values, but many null values are allowed in the column

	It is table level constraint						
check(condition)	while storing value in a column if you want to check some condition, if condition is true, then only insert the value in the table It is table level constraint						
not null	null values are not allowed, it is field level constraint						
default	when user do not enter the value then default value gets added, instead of null, field level constraint						
foreign key	to maintain correctness of data, if the column refers some other column of different or same table with this constraint we can use 2 more constraints on delete <action> on update <action> values of action can be <table border="1"> <tr> <td>set null</td><td>changes done in parent table, then corresponding values should set to null in child table</td></tr> <tr> <td>cascade</td><td>the change made in parent table, the same change should be done in child table</td></tr> <tr> <td>no action</td><td>if we donot add on delete and on update statements, the it is no action by default</td></tr> </table>	set null	changes done in parent table, then corresponding values should set to null in child table	cascade	the change made in parent table, the same change should be done in child table	no action	if we donot add on delete and on update statements, the it is no action by default
set null	changes done in parent table, then corresponding values should set to null in child table						
cascade	the change made in parent table, the same change should be done in child table						
no action	if we donot add on delete and on update statements, the it is no action by default						

Create a table category

Category(cid, cname,cdesc)

cid	Primary key	Number
Cname	Not null	string
Cdsec		string

Create table product to store pid , pname,qty (qty>0), price (default value 10),mfgdate, expdate, cid (foreign key)

Pid	Primary key	Number
Pname	Unique not null	string
Qty	Qty>0	Number
Price	Default 10	Number
Mfgdate		Date
Expdate	Expdate>mfgdate	date
catid	Foreign key reference cid of Category	number

<pre>Create table category(cid int primary key, cname varchar(20) not null, cdesc varchar(20));</pre>	<pre>Create table product(Pid int primary key, Pname varchar(20) not null, qty int check(qty>0), Price double(12,2) default 10, Mfgdate date, Expdate date, catid int, constraint ch_dt check(expdate>mfgdate), constraint fk_cid foreign key(catid) references category(cid));</pre>
---	--

```
insert into category values(1,'chips','very crispy');
insert into category values(2,'cold drink','thanda thanda');
insert into category values(3,'chocolate','yummy');
insert into category values(4,'snacks','very tasty');
```

```
insert into product values(11,'lays',30,40,'2026-09-04','2026-12-06',1);

-----check for primary key cannot be duplicate
insert into product values(11,'lays',30,40,'2026-09-04','2026-12-06',1);
ERROR 1062 (23000): Duplicate entry '11' for key 'product.PRIMARY'

-----check mfgdate>expdate
insert into product values(12,'pringles',30,40,'2026-09-04','2026-08-06',1);
ERROR 3819 (HY000): Check constraint 'ch_dt' is violated.

insert into product values(12,'pringles',30,40,'2026-09-04','2026-12-06',1);

-----check pname cannot be null
insert into product values(13,null,30,40,'2025-10-04','2026-01-06',1);
ERROR 1048 (23000): Column 'Pname' cannot be null

mysql> insert into product values(13,'dates',30,40,'2025-10-04','2026-10-06',4);

-----check the default constraint and foreign key constraint
insert into product(pid,pname,qty,catid) values(16,'nachos',30,10);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`iacsdacaug2026`.`product`, CONSTRAINT `fk_cid` FOREIGN KEY (`catid`)
REFERENCES `category` (`cid`))

-----check that value of price =10 which is default value
mysql> insert into product(pid,pname,qty,catid) values(16,'nachos',30,1);
insert into product(pid,pname,qty,catid) values(15,'fruitandnut',30,3);
```

Create following tables

Course(cid, cname,duration, capacity,rid, fid)

cid	Primary key	Int
Cname	Not null	String
Duration	>0	Int
Capacity	>10	Int
Rid	Foreign key references room(roomid)	Int
Fid	Foreign key references faculty(facid)	int

Faculty(facid,fname,sp_sub)

Facid	Primary key	Int
Fname	Unique Not null	String
Sp_sub		string

Room(roomid,rname,rloc)

roomid	Primary key	Int
rname	Unique Not null	String
rloc		string

Create table room(Roomid int primary key, Rname varchar(20) not null unique, Rloc varchar(20));	Create table course(cid int primary key, cname varchar(20) not null, duration int check(duration>0), capacity int check(capacity >0), rid int, fid int, constraint fk_rid foreign key(rid) references room(roomid) on delete set null on update cascade, constraint fk_fid foreign key(fid) references faculty(facid) on delete set null on update cascade)
Create table faculty(Facid int primary key, Fname varchar(15) unique not null, Sp_sub varchar(15));	

Room

10	Jasmin	1 st floor
11	Rose	1 st floor
12	Mogra	2 nd floor
13	Lotus	2 nd floor

Insert into room values(10,'jasmin','1st floor'),(11,'rose','1st floor'),(12,'mogra','2nd floor'),(13,'lotus','2nd floor');

Faculty

100	Rama	Java
101	Rajesh	Python
102	Asha	CPP
103	Sangeeta	DBMS

Insert into faculty values(100,'rama','java'),(101,'rajesh','python'),(102,'Asha','CPP'),(103,'Sangeeta','DB MS');

Course

1000	Ac	300	240	13	103
1001	Bda	250	60		101
1002	Itiss	240	60	12	
1003	AI	280	100		

Insert into course values(1000,'AC',300,240,13,103),(1001,'bda',250,60,null,101),(1002,'itiss',240,60,12,null),(1003,'AI',280,100,null,null);

3.

Create table employee(eid, name,mobile).

Create table project(pid,pname,duration).

Create table emp_proj(empid,projid,jdate,alloc_time)

Create table employee(eid int primary key,ename varchar(20),mobile varchar(15) unique);	Create table emp_proj(Pid int, Eid int, Jdate date, Alloc_time int, Primary key(pid,eid), Constraint fk_pid foreign key(pid) references project(pid) On delete cascade On update cascade, Constraint fk_eid foreign key(eid) references employee(eid) On delete cascade On update cascade)
---	--

Create table project(pid int primary key, Pname varchar(20),duration int);	
---	--

Eid	Ename	mobile
12	Rajani	222222
13	Manik	333333

Insert into employee values(12,'Rajani',22222),(13,'Manik',333333);

Pid	Pname	Duration
1000	Banking	3
1001	Inssurance	4
1002	Automobile	5

Insert into project values(1000,'Banking',3),(1001,'Inssurance',4),(1002,'Automobile',5);

Pid	Eid	Jdate	Alloctome
1002	12	2025-11-11	20
1001	12	2024-11-11	22
1002	13	2025-11-11	20
1001	13	2024-11-11	22

Create a table mydept, deptno, dname,dloc

Create table mydept(
deptno int primary key,
dname varchar(20) not null,
dloc varchar(20)
)

Create a table myemp
Create table myemp(
Empno int primary key,
Ename varchar(25),
Email varchar(20),
Sal double(12,2) check(sal>=2000),
Comm double(12,2) default 100,
dno int,
hiredate date,
Constraint fno foreign key(Dno) references mydept(deptno)

On delete set null
On update cascade

)

create student table to store student id, name, mobile, and age
name cannot be blank, mobile cannot be blank, and no duplicate values
age should be in range 20 to 100

create course table to store course id, cname, duration of the course, capacity of the course, course capacity should be always < 300

create stud_course table to store
sid, cid marks

create table student(sid int primary key, sname varchar(20) not null, mobile int unique not null, age int check(age between 20 and 100))	create table stud_course(stuid int, coursid int, marks int check(marks between 0 and 100), primary key(stuid,coursid), constraint fk_sid foreign key(stuid) references student(sid) on delete cascade on update cascade, constraint fk_cid foreign key(coursid) references course(cid) on delete cascade on update cascade)
create table course(cid int primary key, cname varchar(20) unique not null, duration int, capacity int check(capacity <=300))	

Sid	Sname	Mobile	Age
15	Rohan	1111	20
20	Rajasi	2222	21

cid	Cname	duration	Capacity	Fees
100	Java	90	100	40000
102	Database	95	70	50000
105	C++	40	20	60000

Sid	Courseid	marks
15	100	95
20	100	97
15	102	96

to delete the table

drop table <tablename>
drop table student;

to delete record

```
-----delete all the rows
Delete from emp

-----delete rows which satisfies the salary condition
Delete from emp
Where sal>2000
```

To update record

```
-----modify ename,sal,job fro all rows
Update emp
Set ename='xxxx',sal=4567,job='manager'

-----modify ename,sal,job fro all rows with sal>2000

Update emp
Set ename='xxxx',sal=4567,job='manager'
Where sal>2000;

-----modify ename,sal,job fro employee with id 7902

Update emp
Set ename='xxxx',sal=4567,job='manager'
Where empno=7902;
```

```
create table mytable(
id int primary key auto_increment,
name varchar(20));
```

```
insert into mytable values(default,'xxxx')
insert into mytable(name) values( 'xxxx')
```

```
to modify start value
alter table mytable auto_increment=1000;
```

auto_increment ---- will generate unique values automatically, it can be used only with int columns, and on primary key which has only single column.

1. Temporary table

```
Create temporary table mytab1(
    Id int primary key,
    Name varchar(20),
    Address varchar(20)
)
```

DML statements

all DML statements work on single table

to insert record	insert into <table name> values(<list of values>) insert into <table name>(<list of columns>) values(<list of values>) insert into <table name>(<list of columns>) values(<list of values>),(<list of values>),(<list of values>)
to delete record if we skip where clause then it will delete all records, but the empty table will remain	delete from <table name> where <condition>
to update data if we skip where clause then it will update all records	update <table name> set col1=val1,col2=val2,col3=val3 where <condition>

Truncate table

truncate table mytable;

it will delete all records but will not delete the table, so empty table will remain

Truncate Vs Delete

truncate is DDL statement, DDL statements are autocommitted	Delete is DML statement
we cannot rollback the changes	changes can be rollback
where clause cannot be used	where clause can be used
If we delete all values using truncate, and if the table has auto_increment column the it will start autogenerated value always from 1	If we delete all values using delete from a table , and if the table has auto_increment column the it will start autogenerated value always from previous maximum value+1

Alter table

To modify table structure we use alter table

add column	-----add column after id column alter table mytable add addr varchar(20) not null after id; -----add column before first column alter table mytable add srno int first; -----add column at the end alter table mytable add mobile varchar(20);
------------	---

	---add multiple columns alter table mytable add email varchar(20) after name, add city varchar(20)
delete a column	alter table mytable drop addr;
modify column definition	-----to change existing columns data type or to add new constraints use modify alter table mytable modify name varchar(50)
modify column name	alter table mytable change name newname varchar(20)
add constraint	1. add a primary key constraint alter table mytable add primary key(id) 2. add foreign key alter table mytable add constraint fk_1 foreign key(location) references mylocation(lid)
drop constraint	alter table mytable drop primary key alter table mytable drop foreign key fk_1
rename the table	alter table mytable rename to mytable1;
can be used in auto_increment	alter table mytable auto_increment=1000;

Drop primary constraint from course table

Alter table course

Drop primary key

Drop foreign key constraint from course table;

Alter table course

Drop foreign key fk_fid;

Nested query

1. simple nested query--- in these types of queries, child query gets executed only once and then outer query will get executed.
 If the child query is independent query then it is called as simple nested query
 Child query is independent query, It is not dependent on parent query
2. co-related query--- in these types of queries, child query gets executed once for each row in the outer query table.
 If the child query is dependent on parent query then it is called as correlated nested query

If the child query is not dependent on outer query, then it is called as simple nested query
but if child query is dependent on outer query, then it is called as co-related query.

1. Find all employees who works in smiths department

```
Select * from emp
Where deptno=(select deptno from emp
              Where ename='smith');
```

2. Find all employees whose job is same as millers job

```
Select * from emp
Where job=(select job from emp
           Where ename='miller');
```

3. Find all employee whose job is same as either miller's job or blake's job

```
Select * from emp Where job in (select job from emp
                                Where ename in ('miller','blake'));
```

4. find all employees who are working in smith's or allen department

```
select * from emp
where deptno in (select deptno
from emp
where ename in ('SMITH','allen'))
```

5. list all employees who are working in accounts department

```
Select * from emp
Where deptno=(Select deptno from dept
              Where dname='Accounting');
```

6. write query to find all employees who works under same mgr as mgr of allen or miller

```
Select * from emp
Where mgr in (select mgr from emp
             Where ename in ('allen','miller'));
```

7. display all employees with sal > allen's salary

```
Select * from emp
Where sal >(select sal from emp
           Where ename='allen')
```

8. find all employees whose sal > either smith's or allen salary

```
select * from emp
where sal > any (select sal
from emp
where ename in ('SMITH','allen'))
```

9. find all employees whose sal > both smith's and allen salary

```
select * from emp
where sal > all (select sal
from emp
where ename in ('SMITH','allen'))
```

10. find all employees with sal < avg salary of department 10

```
Select * from emp
Where sal < (select avg(sal) from emp
Where deptno=10);
```

11. find all employees with sal < avg salary of either department 10 or department 20

```
Select * from emp
Where sal < any (select avg(sal) from emp
Where deptno in (10,20)
Group by deptno);
```

12. find all employees with sal > avg salary of either clerk or manager

```
Select * from emp
Where sal > any (select avg(sal) from emp
Where job in ('clerk','manager')
Group by job);
```

13. find all employees with sal < avg salary of accounting department.

```
select * from emp
where sal < (select avg(sal) from emp where deptno =(select deptno
from dept
where dname='accounting'));
```

14. find all employees with sal > min sal of either miller's department or jones department

```
Select * from emp
Where sal > any (select min(sal) from emp
Where deptno in (select deptno
```

```
From emp
Where ename in
('miller','jones'))
Group by deptno);
```

15. find all employees with sal > average sal of its own department.

```
select * from emp e
where e.sal > (select avg(sal) from emp m where m.deptno=e.deptno )
```

16. write a query to find all employees with sal > min sal of all employees who are working under his own manager

```
Select * from emp e
Where sal > (select min(sal)
From emp m
Where m.mgr=e.mgr)
```

in correlated query usually we use 2 operators

1. exists--→ it will return true if child query returns one or more rows in the o/p, false otherwise
2. not exists---→ it will return true if child query does not return any rows in the o/p, false otherwise.

17. Find all departments in which no employees are there

```
Select * from dept d
Where not exists (select *
From emp m
Where m.deptno=d.deptno)
```

18. Find all departments in which some employees are there

```
Select * from dept d
Where exists (select *
From emp m
Where m.deptno=d.deptno)
```

19. Find all employee who are mgr for some employees

```
Select * from emp e
Where exists (select *
From emp m
Where m.mgr=e.empno)
```

20. Find all employee who are not mgr for some employees

```

Select * from emp e
Where not exists (select *
                  From emp m
                  Where m.mgr=e.empno)

```

12. list all employees who are mgr of some employees, and their sal > avg sal of dept 20, and job is 'manager'

```

Select * from emp e
Where job='manager' and sal > (select avg(sal) from emp where deptno=20) and exists
(select *
  From emp m
  Where m.mgr=e.empno)

```

course

cid	Cname	Fid	Rid	Duration
1000	AC	102	10	360
2000	BDA	101		300
3000	VLSI			270

Room

Roomid	Rname	rlocation
10	Lotus	1 st floor
20	Vinca	2 nd floor
30	Jasmin	1 st floor

Facid	Fname	Sp.subject
12	Rita	Java
100	Rama	ML
101	Sanjay	C++
13	Raju	C++

1. find all the rooms which can be assigned to BDA

```

select * from room r
where not exists (select * from course c where c.rid=r.roomid)

```

2. find all the rooms which can be assigned to BDA, but the room should be on the 2nd floor

```

select * from room r
where rlocation='2nd floor' and not exists (select * from course c where c.rid=r.roomid)

```

3. display coursename of the course assigned to Rama
 Select * from course
 Where fid=(select facid from faculty where fname='Rama');
4. find courses for which no room is assigned.
 Select * from course c where rid is null;

Nested queries in DML operations

1. update smiths salary to allen's salary

```
update emp
set sal=(select e.sal from (select * from emp) e where e.ename='allen')
where ename='SMITH' ;
```

2. delete all rows of employees who works in same department as allen

```
delete from emp
where deptno=(select deptno from (select * from emp ) e
              where e.ename='ALLEN');
```

Note: when you create a copy of table using nested query, then constraint will not get copied, only structure and data will get copied

3. create a copy of emp table

```
create table emp1
as
(select * from emp);
```

```
----to copy only empno,ename and
sal
create table emp13 as
(select empno,ename,sal
from emp);
```

5. create a copy of emp table but copy only empno, name and deptno

```
Create table emp2
As
(select empno,ename,deptno
From emp)
```

6. create a copy of emp table but copy only empno, name and deptno, do not copy any data

```
Create table emp2
As
(select empno,ename,deptno
From emp where 1=2
)
```

7. insert data of all employees from department 20, in emp2 table

```
insert into emp2
(select empno,ename,deptno from emp
Where deptno=20)
```

8. find all employees with sal >= smiths's salary and <= blake's salary

```
Select * from emp
Where sal between (select sal from emp where ename='smith') and (select sal
From emp where ename='blake');
```

9. list all products with category chips

```
Select * from product where catid=(select cid from category where cname='chips');
```

10. list all products with qty < avg quantity of chocolate type products

```
Select * from product where qty<(select avg(qty) from product where catid =
(select cid from category where cname='chocolate'))
```

11. list all categories for which no products are there

```
Select * from category c
Where not exists (select * from product p where p.catid=c.cid)
```

12. list all products for which no category is assigned

```
Select * from product where catid is null;
```

13. list all products of chocolate category if the price>30

```
Select * from product where price > 30 and catid = (select cid from category
Where cname='chocolate')
```

14. list all products with price < avg price of its own category

```
Select * from product p where price<(select avg(price) from product q where
q.catid=p.catid)
```

15. list all products with price not >= price of nachos and <= price of lays

```
Select * from product where price not between (select price from product where
pname='nachos') and (select price from product where pname='lays');
```

16. list all products with category is either chips or chocolates

```
Select * from product where catid in (select cid from category  
Where cname in ('chips','chocolate');
```

17. list all categories for which products are there and cdesc contains Thanda

```
Select * from category c where c.cdesc like 'thanda' and exists (select * from product  
p where p.catid=c.cid);
```

18. list all products with chips categories and qty > avg qty of its own category

```
Select * from product p where catid=(select cid from category c where  
c.cname='chips') and qty > (select avg(qty) from product q where q.catid=p.catid)
```